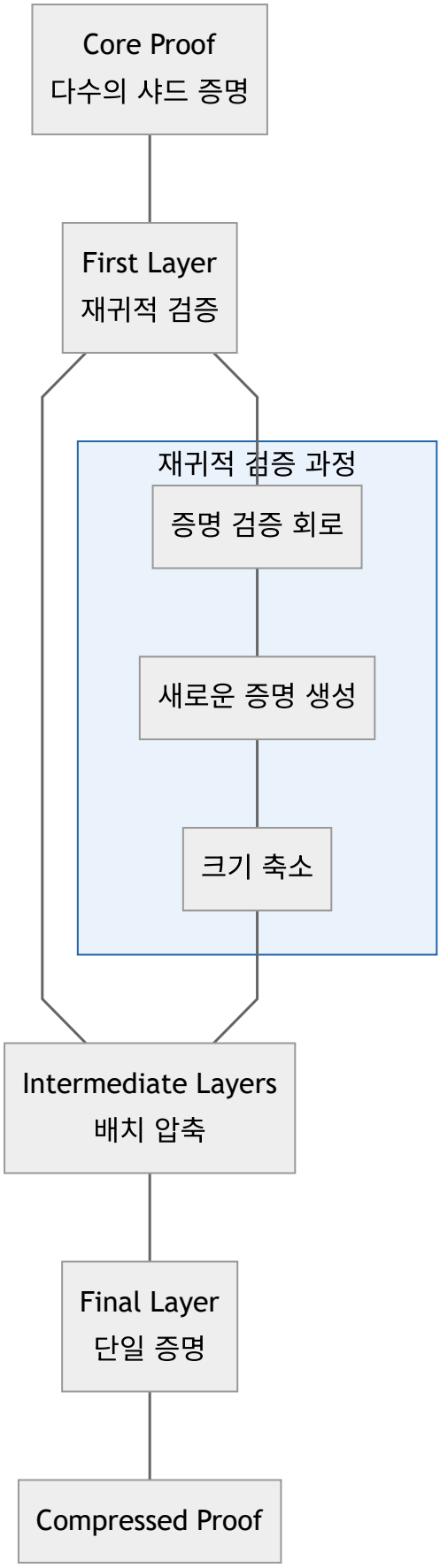
07. 압축 및 재귀적 증명 과정 분석 (Compression and Recursive Proof Process Analysis)

개요

SP1의 압축 및 재귀적 증명 과정은 여러 개의 샤드 증명을 하나의 증명으로 통합하고, 증명 크기를 최적화하는 핵심 단계입니다. 이 문서는 압축 과정, 재귀적 증명 생성 알고리즘, 그리고 증명 크기 최적화 기법을 상세히 분석합니다.

1. 압축 과정 전체 아키텍처

1.1 압축 파이프라인 구조



1.2 압축 단계별 구성

SP1의 압축 과정은 다음과 같은 계층적 구조를 가집니다:

1. **First Layer**: 개별 샤드 증명들을 재귀적으로 검증
2. **Intermediate Layers**: 배치 단위로 증명들을 압축
3. **Final Layer**: 최종 단일 증명 생성

2. compress 함수 상세 분석

2.1 함수 시그니처 및 매개변수

```
pub fn compress(
&self,
vk: &SP1VerifyingKey,
proof: SP1CoreProof,
deferred_proofs: Vec<SP1ReduceProof<InnerSC>>,
opts: SP1ProverOpts,
) -> Result<SP1ReduceProof<InnerSC>, SP1RecursionProverError>
```

주요 매개변수:

- `vk` : 검증 키
- `proof` : 압축할 Core Proof (다수의 샤드 증명 포함)
- `deferred_proofs` : 지연된 증명들
- `opts` : 증명 생성 옵션

2.2 배치 크기 및 트리 높이 계산

```
let batch_size = REDUCE_BATCH_SIZE; // 2
```

```
let first_layer_batch_size = 1;


let shard_proofs = &proof.proof.0;
let first_layer_inputs = self.get_first_layer_inputs(
vk,
shard_proofs,
&deferred_proofs,
first_layer_batch_size
);


// 트리 높이 계산
let mut expected_height = if first_layer_inputs.len() == 1 { 0 } else { 1 };
let mut num_layer_inputs = first_layer_inputs.len();
while num_layer_inputs > batch_size {
num_layer_inputs = num_layer_inputs.div_ceil(2);
expected_height += 1;
}
```

트리 구조 최적화:

- 첫 번째 레이어는 배치 크기 1로 개별 처리
- 이후 레이어들은 배치 크기 2로 이진 트리 구조 형성
- 높이 계산을 통한 메모리 사용량 예측

2.3 병렬 처리 아키텍처

압축 과정은 다음과 같은 다단계 병렬 처리를 사용합니다:

스레드 구조:

1. **입력 전송 워커**: 첫 번째 레이어 입력을 채널로 전송
2. **레코드/추적 생성 워커**: 재귀 회로 실행 및 추적 생성
3. **증명 생성 워커**: STARK 증명 생성

```
// 입력 전송 워커
let (input_tx, input_rx) = sync_channel::<(usize, usize, SP1CircuitWitness, bool
opts.recursion_opts.checkpoints_channel_capacity,
);


// 레코드/추적 생성 워커들
for _ in 0..opts.recursion_opts.trace_gen_workers {
s.spawn(move || {
// 재귀 회로 실행 및 추적 생성
});
}


// 증명 생성 워커
s.spawn(move || {
// STARK 증명 생성
});
```

3. 재귀적 증명 생성 알고리즘

3.1 재귀 회로 타입

SP1은 세 가지 주요 재귀 회로 타입을 사용합니다:

```
pub enum SP1CircuitWitness {
Core(SP1RecursionWitnessValues<CoreSC>),
Deferred(SP1DeferredWitnessValues<InnerSC>),
Compress(SP1CompressWitnessValues<InnerSC>),
}
```

각 타입의 역할:

- **Core**: 샤드 증명들을 재귀적으로 검증
- **Deferred**: 지연된 증명들을 처리
- **Compress**: 여러 증명을 하나로 압축

3.2 SP1RecursionWitnessValues 구조

```
pub struct SP1RecursionWitnessValues<SC: StarkGenericConfig> {
pub vk: StarkVerifyingKey<SC>,
pub shard_proofs: Vec<ShardProof<SC>>,
pub leaf_challenger: SC::Challenger,
pub shard_challenges: Vec<SC::Challenger>,
pub initial_reconstruct_challenger: SC::Challenger,
pub is_complete: bool,
```

```
pub is_first_shard: bool,
pub reconstruct_deferred_digest: [SC::Val; 8],
}
```

주요 필드 설명:

- `vk` : 검증에 사용할 검증 키
- `shard_proofs` : 검증할 샤드 증명들
- `leaf_challenger` : 리프 노드 챌린지
- `shard_challenges` : 각 샤드별 챌린지
- `is_complete` : 완료 여부 플래그
- `reconstruct_deferred_digest` : 지연된 다이제스트 재구성

3.3 재귀 회로 실행 과정

```
// 프로그램 및 위트니스 스트림 생성
let (program, witness_stream) = match input {
SP1CircuitWitness::Core(input) => {
let mut witness_stream = Vec::new();
Witnessable::<InnerConfig>::write(&input, &mut witness_stream);
(self.recursion_program(&input), witness_stream)
}
// ... 다른 타입들
};
```

```
// 재귀 런타임 실행
let record = {
let mut runtime = RecursionRuntime::<Val<InnerSC>, Challenge<InnerSC>, _>::new
program.clone(),
self.compress_prover.config().perm.clone(),
);
runtime.witness_stream = witness_stream.into();
runtime.run().unwrap();
runtime.record
};
```

실행 단계:

1. **위트니스 직렬화**: 입력 데이터를 바이트 스트림으로 변환
2. **프로그램 선택**: 입력 타입에 따른 적절한 재귀 프로그램 선택
3. **런타임 실행**: 재귀 회로 실행 및 레코드 생성
4. **의존성 생성**: 증명 생성에 필요한 의존성 계산

4. 압축 최적화 기법

4.1 프로그램 캐싱

SP1은 재귀 프로그램의 컴파일 비용을 줄이기 위해 LRU 캐시를 사용합니다:

```
pub lift_programs_lru: Mutex<LruCache<SP1RecursionShape, Arc<RecursionProgram<B
```

```
pub join_programs_map: BTreeMap<SP1CompressWithVkeyShape, Arc<RecursionProgram<
```

캐싱 전략:

- **Lift Programs**: LRU 캐시로 동적 관리
- **Join Programs**: BTreeMap으로 정적 관리
- **캐시 미스 추적**: 성능 모니터링을 위한 카운터

4.2 배치 처리 최적화

```
let chunked_records = chunk_vec(records, opts.shard_batch_size);

let chunked_main_traces = chunk_vec(main_traces, opts.shard_batch_size);
```

배치 처리 이점:

- 메모리 사용량 제어
- 병렬 처리 효율성 향상
- 캐시 지역성 개선

4.3 동기화 최적화

```
let record_and_trace_sync = Arc::new(TurnBasedSync::new());
```

TurnBasedSync 특징:

- 순서 보장이 필요한 작업의 효율적 동기화
- 데드락 방지
- 공정한 스레드 스케줄링

5. SP1CompressWitnessValues 분석

5.1 구조 정의

```
pub struct SP1CompressWitnessValues<SC: StarkGenericConfig> {
pub vks_and_proofs: Vec<(StarkVerifyingKey<SC>, ShardProof<SC>)>,
pub is_complete: bool,
}
```

필드 설명:

- `vks_and_proofs` : 압축할 (검증키, 증명) 쌍들
- `is_complete` : 압축 완료 여부

5.2 압축 과정에서의 역할

```
let input = SP1CircuitWitness::Compress(SP1CompressWitnessValues {
vks_and_proofs,
is_complete,
});
```

압축 단계별 사용:

1. **중간 레이어**: 여러 증명을 하나로 압축
2. **최종 레이어**: 완전한 압축 증명 생성
3. **검증**: 압축된 증명의 유효성 확인

6. 머클 증명 통합

6.1 SP1CompressWithVKeyWitnessValues

```
pub struct SP1CompressWithVKeyWitnessValues<SC: StarkGenericConfig + FieldHashe
pub compress_val: SP1CompressWitnessValues<SC>,
pub merkle_val: SP1MerkleProofWitnessValues<SC>,
}
```

6.2 머클 증명 생성

```
pub fn make_merkle_proofs(
&self,
input: SP1CompressWitnessValues<CoreSC>,
) -> SP1CompressWithVKeyWitnessValues<CoreSC> {
let num_vks = self.recursion_vk_map.len();
// 머클 트리에서 증명 생성
}
```

머클 증명의 역할:

- 검증 키의 유효성 증명
- 허용된 VK 집합에 대한 멤버십 증명
- 보안성 강화

7. 성능 최적화 전략

7.1 메모리 관리

지연 삭제:

```
rayon::spawn(move || {
drop(record);
});
```

채널 용량 조정:

```
sync_channel::<(usize, usize, SP1CircuitWitness, bool)>(
opts.recursion_opts.checkpoints_channel_capacity,
);
```

7.2 병렬성 최적화

워커 스레드 수 조정:

```
for _ in 0..opts.recursion_opts.trace_gen_workers {
// 워커 스레드 생성
}
```

작업 분산:

- CPU 코어 수에 따른 워커 수 조정
- I/O 바운드 작업과 CPU 바운드 작업 분리

- 메모리 사용량과 처리 속도 간 균형

7.3 형태 최적화

```
if let Some(shape_config) = shape_config {
for record in records.iter_mut() {
shape_config.fix_shape(record).unwrap();
}
}
```

형태 고정의 이점:

- 예측 가능한 메모리 사용량
- 증명 크기 일관성
- 검증 시간 최적화

8. 오류 처리 및 복구

8.1 오류 타입

```
pub enum SP1RecursionProverError {
RuntimeError(String),
InvalidProof,
// ... 기타 오류 타입들
}
```

8.2 복구 전략

재시도 메커니즘:

- 일시적 오류에 대한 자동 재시도
- 백오프 전략을 통한 부하 조절
- 최대 재시도 횟수 제한

상태 복구:

- 체크포인트를 통한 중간 상태 복구
- 부분 결과 재사용
- 점진적 복구 전략

9. 압축 효율성 분석

9.1 압축 비율

이론적 압축 비율:

- N개 샤드 → 1개 증명: O(log N) 크기 감소
- 재귀 깊이에 따른 압축 효과
- 배치 크기 최적화를 통한 효율성 향상

9.2 성능 메트릭

측정 지표:

- 압축 시간 vs 샤드 수
- 메모리 사용량 vs 배치 크기
- 증명 크기 vs 재귀 깊이

최적화 목표:

- 압축 시간 최소화
- 메모리 사용량 제어
- 최종 증명 크기 최소화

10. 결론

SP1의 압축 및 재귀적 증명 과정은 다음과 같은 핵심 특징을 가집니다:

1. **계층적 압축**: 트리 구조를 통한 효율적 압축
2. **병렬 처리**: 다단계 파이프라인을 통한 성능 최적화
3. **캐싱 전략**: 프로그램 재사용을 통한 컴파일 비용 절약
4. **메모리 최적화**: 스트리밍 처리를 통한 메모리 효율성
5. **유연성**: 다양한 압축 전략 지원

이러한 설계를 통해 SP1은 대규모 프로그램에 대해서도 실용적인 크기의 증명을 생성할 수 있으며, 검증 시간을 크게 단축시킬 수 있습니다.